RailCall — HIPAA Security Risk Analysis (v1)

Date written: 2026-07-21 **Applies to:** - `railcall-engine` @ `9521956fd949d956e315edb74dc434c284f4526f` (branch `feat/paid-tier-entitlement`) - `railcall-core` @ `06bdd1449eddd6b65438ca92f53273e7f3334cf6` (branch `main`) **Status:** ADOPTED 2026-07-21 by Sami Ben Chaalia (Security Officer). **Framework:** HHS SRA Tool categories aligned to 45 CFR §164.308(a)(1)(ii)(A) (Security Rule Risk Analysis). **Scope of this SRA:** the RailCall product surface — engine, station (local runtime), gateway (railcall-core.onrender.com), CLI, receipts and audit chain. Personal, HR, marketing, and back-office systems are OUT of scope for v1.

---

0. About this document

This SRA is written to be read, not signed off. Every implemented control cites the file and function that implements it. Every gap is stated as a gap with the reason and the mitigation plan. Nothing is claimed as "in progress" without a specific owner and a date.

This is version 1. It should be re-verified against the code at each substantive release (station-v0.16 was the release this document was written against) and re-adopted at least annually per §164.316(b)(2)(iii).

Two things this SRA is deliberately NOT: - **Not a compliance certification.** HIPAA has no certifying body; compliance is self-determined. Publishing this document does not make RailCall compliant. - **Not a substitute for organizational risk management.** Administrative and physical safeguards (§164.308, §164.310) are covered here at the level a small-team product can honestly claim; a healthcare deployment requires the customer's own §164.308(a)(1)(ii)(B) implementation on top.

---

1. Scope — the systems in play

System	What it does	Where PHI could enter
Local engine (`railcall-engine`)	User's machine, runs governed workflows on `127.0.0.1`. Signs receipts.	Any workflow field the user configures.
Station bundle (`railcall_station.tar.gz`, released via installer)	Ships the engine + workbench code to end users.	N/A (no runtime data at rest).
CLI (`railcall_cli.py`, `railcall_companion_daemon.py`)	Local commands (`railcall demo`, `railcall activate`), signs `railcall demo` receipts, holds the vault.	Not by default — the CLI does not accept clinical data.
Gateway (`cloud_gateway.py` on Render at `https://railcall-core.onrender.com`)	Mint entitlements, seat validation, Stripe webhook, meter/balance. Sees `sha256(api_key)`, `install_pubkey`, integrity HASHES from receipt bundles — **never receipt bodies**.	Contract violation only — see §5.3 for what forces this to be true.
Website + docs (`railcall-contrib/website-v2`, deployed to `railcall.ai`)	Marketing + docs. No PHI, no PII beyond signup email.	Signup emails only.

Key architectural fact. The engine executes locally under the customer's control. RailCall the operator never sees the customer's workflow inputs or outputs; the gateway only ever receives:

1. `sha256(api_key)` for billing (`cloud_gateway.py` /v1/meter, /v1/seat/checkin — the `key_hash` path is the primary; raw keys are a legacy alternative kept for backward compatibility, both paths flow to the same DB column)
2. Signed integrity hashes and attestation ids (`cloud_gateway.py` /v1/attestation/countersign)
3. Stripe webhook data (payment status, subscription lifecycle)

This means for the majority of RailCall deployments, **RailCall is not a business associate at all** (§160.103) because we do not create, receive, maintain, or transmit PHI on behalf of the covered entity. When we ARE a business associate (e.g. a hosted feature that ships PHI intentionally), the BAA in `legal/BAA_DRAFT.md` applies and Exhibit A of that BAA maps every data flow explicitly.

---

2. Inventory of ePHI (§164.308(a)(1)(ii)(A))

Where ePHI could be created, received, maintained, or transmitted by systems in scope:

Location	Type	Retention	Encryption at rest	Encryption in transit
Local vault (`~/.railcall/keys.local.json`, `<station>/.railcall_workspace/keys.local.json`)	BYOK credentials + Ed25519 signing seed	Until user deletes	OS keychain (macOS) / DPAPI (Windows) / Secret Service (Linux) via `seed_store.py`; degraded plaintext-file only on non-macOS if the OS backend refuses (`seed_store.status()` reports honestly)	N/A (never transmitted)
Local receipts (`~/.railcall/receipts/`)	Signed per-action records — RailCall's OWN execution records, NOT customer clinical data	Until user deletes	0600 file permissions	N/A (never transmitted by us)
Local audit chain (`workbench/audit_chain.py` + on-disk JSONL)	Actor + event trail	Until user deletes	0600, hash-chained, Ed25519 signed	N/A
Gateway DB (`consumers`, `seat_reservations`, `processed_events` in `railcall-core`'s Postgres/SQLite)	Email, `sha256(api_key)`, seat counts, install pubkeys, Stripe customer ids	Until account deletion	Encrypted-at-rest per Render's managed Postgres (their responsibility surface, cited from Render docs)	TLS 1.3 to gateway
Gateway logs (Render service logs)	Same as DB writes + operational logs. Email is redacted from audit log per P0-4 remediation.	~30 days on Render	Render-controlled	Render-controlled

Where ePHI does NOT go, by design: - Never into any receipt body signed by us — enforced by `phi_guard.py` PHI detection + `approval_airlock.py:redact()` (see §5 for the closed egress path) - Never into the gateway request body — /v1/seat/checkin and /v1/meter both reject bodies containing anything except the three declared fields; /v1/attestation/countersign accepts only a hash + id - Never into any observability tool RailCall the operator uses — because it never leaves the customer's box

3. Threat & vulnerability catalogue (§164.308(a)(1)(ii)(A) sub-a)

Structured as: threat → attack path → what today's controls do → residual risk.

3.1 Local file-system exposure (P0-1)

- **Threat:** Copying, backing up, or synchronising `keys.local.json` to a location the user does not intend (Time Machine, iCloud, Dropbox, `git add`).
- **Attack path:** The signing seed's presence in that file lets any reader forge Ed25519 receipts that verify cleanly against the install's published pubkey, undermining every downstream trust claim.
- **Control (implemented):** `seed_store.py` moves the seed into the OS keychain on boot (macOS `security(1)` login keychain; Windows DPAPI; Linux libsecret via `secret-tool`). Migration is idempotent and fail-safe (keychain-write must succeed BEFORE the plaintext copy is removed — `test_seed_store.py:5b` asserts this).
- **Verification:** `test_seed_store.py` (16 checks, all pass, on both engine and CLI halves).

- **Residual risk:** The keychain protects against copying/leakage, NOT against malware running as the same OS user. Documented in `seed_store.py:_BACKEND_NOTE` and stated in the tool's own status output. Mitigation requires hardware attestation, which is customer-side.

3.2 Audit-log tamper (P0-2)

- **Threat:** A local process rewrites `audit_log.jsonl` / `integration_audit.jsonl` to remove evidence of an incident.
- **Attack path:** Post-incident inspection reads a tampered log and reports "no anomalies," meeting §164.308(a)(1)(ii)(D) audit requirement in name but not in substance.
- **Control (implemented):** `audit_chain.py` (in `railcall-engine/workbench/`) gives every audit record a `prev_hash` linking it to the previous record, Ed25519-signs the record with the install key, and adds an `actor` field. Both `audit_log()` and `integration_audit()` in `studio_server.py` route through the chain with a plain-append fallback (losing a record is worse than losing a chain).
- **Verification:** `test_audit_chain.py` (17 checks, all pass), and test 4a specifically asserts the LIMIT: tail-truncation by the install-key holder stays undetectable (chain "intact", count under-reports). `verify()` returns that caveat inside its own result so callers can't accidentally over-claim.
- **Residual risk:** Tail-truncation by the key-holder. Mitigation: off-box witness (periodic head anchoring to a service the operator does not control). This is on the roadmap — tracked as Phase-1 item 1.13 in `claude-context.md`.

3.3 PHI egress via the "PII firewall" (P0-3)

- **Threat:** A workflow field labelled generically (e.g. `notes`, `field1`) contains clinical narrative — patient name in free text, diagnosis, MRN.
- **Attack path (pre-remediation):** The pre-P0-3 `redact()` only matched field names against `SECRET_HINT = ("token", "key", "secret", "password", "apikey", "authorization", "bearer", "dsn")`. A field named `notes` passed through verbatim into the receipt payload and — if the workflow called a cloud model — into that model's request body. Verified by execution: SSN survived redaction unchanged.
- **Control (implemented):** `phi_guard.py` (in `railcall-engine/workbench/`) detects HIPAA identifiers by both field-name and by value. `approval_airlock.py:redact()` now runs three passes in order: credentials by field name → identifiers by field name → identifiers by value (recursing into nested dicts/lists). Auto-escalation: if any field name is a clinical signal (`phi_guard.CLINICAL_SIGNAL`), the whole payload escalates and unrecognised free text is redacted wholesale.
- **Install-level opt-in (P0-3 completion):** `RAILCALL_PHI_MODE` env var (or `<ws>/phi_mode.json`) declares a whole install PHI-bearing so narrative in ANY generic field is redacted, even without a clinical field name present. Verified: `test_phi_guard.py` 29 checks (was 19 pre-completion), all pass.
- **Verification:** Reproduced end-to-end via the real `airlock_card()` function against `Jane Doe / 1984-03-12 / MRN-889231 / adenocarcinoma / metoprolol / 123-45-6789` payload — none appear anywhere in the card. `api_key` still redacts to `••••••`.
- **Residual risk (documented, NOT hidden):**
- Personal names in free text are not detectable — only escalated-mode wholesale redaction handles them. Test 7a asserts this openly.
- Biometrics, photographs, and §164.514(b)(2)(R) "any other unique identifying number" are not detectable at all.
- Escalation keys off field NAMES. A clinical payload whose fields are all generically named (`field1`, `data`, `text`) will NOT escalate unless the install-level PHI mode is on. Callers handling PHI must set the mode.
- This is redaction, not de-identification. It does NOT produce Safe Harbor de-identified output.

3.4 Plaintext PII in audit records (P0-4)

- **Threat:** Email addresses land in `audit_log.jsonl` in the clear. Verified pre-remediation: 2 of 49 real records contained a plaintext `email` field.
- **Attack path:** In a healthcare context, an email in an audit record is identifying information under HIPAA. Contradicted our own "never logs secret values" guarantee (which covered credentials only).
- **Control (implemented):** `email` added to the sensitive-field list; audit records no longer carry plaintext addresses. Closed in the same commit as the audit-chain (P0-2 remediation, commit `0ac6002b0`).

- **Verification:** Post-fix inspection of the same log corpus → zero plaintext email fields.

3.5 CLI signing seed at rest (P0-1 CLI half — remediated 2026-07-21)

- **Threat:** Same as §3.1 above but for a separate seed file: `~/.railcall/keys.local.json` (used by `receipt_signer.py` via the daemon). Was plaintext 0600 hex until 2026-07-21 despite the station-side P0-1 fix landing in June.
- **Attack path:** Anyone who could read the file could forge the receipts that `railcall demo` produces — the ones users show people.
- **Control (implemented, 2026-07-21):** `railcall-core/seed_store.py` (ported verbatim from engine), routed from `railcall_companion_daemon.py:_ensure_signing_seed()` and `railcall_cli.py:_persist_signing_seed()`. Live migration on the author's workstation proved seed bytes byte-identical after migration, pubkey unchanged, plaintext removed from vault.
- **Verification:** `railcall-core/tests/test_cli_seed_store.py` (8 checks, all pass). Commit `f81e11c`. Ships in station-v0.16.
- **Residual risk:** Same as §3.1 — malware running as the same user can still reach the keychain. Documented in the module and surfaced via `railcall doctor` output (`signing seed at rest: macos_keychain — …`).

3.6 Paid-tier billing integrity

- **Threat:** A customer who cancels their subscription keeps consuming seats indefinitely because the DB row was never updated.
- **Control (implemented, 2026-07-21):** `cloud_gateway.py` now handles `customer.subscription.deleted` and `customer.subscription.updated` (terminal statuses only). Idempotent on event_id. Deactivated accounts can't mint (test 5 asserts this).
- **Verification:** `test_stripe_lifecycle.py` (11 checks, all pass).

3.7 Cross-install entitlement replay

- **Threat:** A paying customer copies their entitlement token to another machine to bypass their seat cap.
- **Control (implemented):** `entitlement.py` binds `install_pubkey` inside the signed body. `verify_entitlement()` checks the token against THIS install's pubkey and returns free-tier on mismatch. A copied token activates NOTHING elsewhere. Additionally, `/v1/seat/checkin` enforces the seat cap server-side against the CURRENT set of active installs.
- **Verification:** `test_activate_e2e.py` step 3a (SAME entitlement, other install → free), and `test_paid_full_e2e.py` E1-E4 (4th install refused with honest posture, nonce not burned so retry-after-free works).

3.8 Loopback-only bind, dual-control approval

- **Threat:** Cross-origin browser attack, remote takeover of the local studio, self-approval by a compromised browser.
- **Control (implemented):** `studio_server.py` binds `127.0.0.1` only (never `0.0.0.0`); a per-process CSRF session token is embedded in the served page and rate-limited; the "approve code" is NEVER templated into any served page — it prints to the launching terminal only, so a compromised browser cannot self-approve external sends or policy changes.
- **Verification:** Read `studio_server.py:12,63,71-88,104-120`. This is architectural, not a specific test — the constant nature of the loopback bind + the terminal-only approve token IS the control.

---

4. §164.312 technical safeguards inventory

Status vocabulary: **IMPLEMENTED** (control exists and is default on) · **PARTIAL** (exists but not default / incomplete) · **GAP** (not implemented).

4.1 §164.312(a)(1) Access Control

Spec	Status	Evidence
Unique user identification (R)	**GAP**	Single-user local model. No per-user identity in the station. `studio_server.py` binds loopback and treats any local caller as the operator. Mitigation for multi-user hosts is on the roadmap; single-workstation shape is defensible today for the current customer profile.
Emergency access procedure (R)	**GAP**	Not implemented. No documented break-glass path. Mitigation: customer's own §164.308(a)(6) incident response covers this at the operational layer.
Automatic logoff (A)	**GAP**	No idle timeout on the studio session. `SESSION_TOKEN` lives for the process lifetime (`studio_server.py:68,107`). Addressable spec; the alternative implemented is the loopback-only bind + short-lived process model.
Encryption/decryption (A)	**IMPLEMENTED**	AES-256-GCM + scrypt vault (`railcall-contrib/vault/vault.py:38-44,95-110`) is opt-in per-integration. Signing seed at rest is protected by `seed_store.py` (§3.1, §3.5).

4.2 §164.312(b) Audit Controls

Spec	Status	Evidence
Record & examine activity (R)	**IMPLEMENTED**	Two append-only JSONL logs (`integration_audit.jsonl`, `audit_log.jsonl`) written via `audit_chain.py` — hash-chained, Ed25519-signed, actor-tagged. See §3.2.

Residual: tail-truncation by the install-key holder is not locally detectable. Off-box witness is the mitigation, not yet implemented.

4.3 §164.312(c)(1) Integrity

Spec	Status	Evidence
Authenticate ePHI (A)	**IMPLEMENTED** for records	Ed25519-signed receipts (`workbench/railcall_signing.py`), independent re-verifying auditor (`workbench/audit_workflow.py:1-20`), hash-chained meter (`primitives/usage_meter.py`), issuer pubkey pinned in `entitlement.py:49` and served at gateway `/v1/issuer/pubkey`.

Caveat: this authenticates RailCall's OWN execution records. It is not a control over customer ePHI, which RailCall never stores. The BAA (Exhibit B item 2) states this explicitly.

4.4 §164.312(d) Person or Entity Authentication

Spec	Status	Evidence
Verify identity (R)	**PARTIAL**	Strong for AGENTS: mandatory Ed25519 signature verification on every request; no passwords or API keys carried across the wire on the paid path (blind hash for seat checkin); failed verify → 401 (`primitives/gateway_auth.py:1-30`).

Gap: no HUMAN authentication. Any local process reaching loopback is trusted as the operator. Defensible on a single-user workstation; not defensible for shared or multi-user hosts, which is exactly the healthcare deployment shape. Multi-user hardening tracked as roadmap.

4.5 §164.312(e)(1) Transmission Security

Spec	Status	Evidence
Integrity controls (A)	**IMPLEMENTED**	Payload hashing + idempotency keys (`approval_airlock.py:40–45`); signed receipts over every external touch.
Encryption in transit (A)	**IMPLEMENTED**	HTTPS to the gateway (`RAILCALL_GATEWAY_URL` → `https://…`); loopback traffic never leaves the host.
PHI scrubbing before egress	**IMPLEMENTED**	See §3.3 (P0-3 remediation), §3.4 (P0-4 remediation).

---

5. What could still fail — residual risks by category

Each risk here is a real one. Each has a stated mitigation OR a stated "documented and accepted" line. This section is what the SRA is FOR — the honest catalogue.

5.1 Same-user malware on the customer's box

The seed_store keychain protection assumes the OS's user boundary holds. A process running as the same user (compromised browser, malicious pip package, etc.) can request the keychain entry and receive it. This is a generic problem for every unattended-signing local tool.

Mitigation options if the customer's threat model requires it: - Hardware token (e.g. YubiKey) instead of keychain — breaks unattended operation. - Per-signature passphrase prompt — breaks unattended operation. Neither is implemented. Documented; accepted as inherent to unattended local signing.

5.2 Tail-truncation of the audit chain by the install-key holder

`test_audit_chain.py:4a` asserts this limit specifically. Mitigation is an off-box witness that periodically anchors the chain head to a service the operator does not control. Not implemented. On the roadmap.

5.3 Contract-violation PHI egress via `/v1/attestation/countersign`

The endpoint's contract is: "we accept only an integrity HASH and an attestation id." Enforced by explicit param typing in `cloud_gateway.py` — the endpoint has no field for "receipt body." A caller could still POST extra fields, but the handler ignores them (FastAPI's `Form(...)` typing does not forward unknown fields to Stripe or to the DB write).

Residual: a future edit that starts accepting a body field could regress this. A code-review rule ("no PHI-shaped fields on gateway request bodies") is the process control; a static scan is the roadmap for automated enforcement.

5.4 Third-party outages / secret loss

- Render is our hosting provider. An outage means the gateway is unavailable — free-tier installs keep working (local engine has no dependency on the gateway); paid installs cannot mint new entitlements but existing entitlements still verify until expiry.
- Stripe outage → new subscriptions can't be created; existing subscriptions continue to be charged by Stripe independently of our service.
- **Issuer seed loss** = catastrophic. Every outstanding entitlement becomes unforgeable-but-unreplaceable; the fix is a full station re-release with a new pinned pubkey. Backup lives in Sami's password manager + offline copy per the ceremony documented in `tools/mint_issuer_keypair.py`.

5.5 Legacy fields on the paid path

`consumers.free_runs_remaining` / `runs_used` are metered-era fields kept for free-tier quota. Paid-tier rows leave them at 0 by convention (enforced in `CONSUMER_SEAT_UPSERT`). A future edit that starts writing non-zero values on a paid row would confuse the metering read path. Schema comment on `CREATE TABLE consumers` documents this invariant.

6. Administrative safeguards (§164.308) — small-team honest posture

A one-person / two-person operation cannot map every §164.308 spec to a formal document. What we DO have:

Spec	Status	How
§164.308(a)(1)(i) Security management process	**PARTIAL**	This SRA is the artifact. Reviewed at each station release.
§164.308(a)(1)(ii)(A) Risk analysis	**IMPLEMENTED**	This document.
§164.308(a)(1)(ii)(B) Risk management	**PARTIAL**	Each identified gap has an owner (Sami) and a mitigation path. No formal risk register beyond this file.
§164.308(a)(1)(ii)(D) Information system activity review	**IMPLEMENTED** at customer install; **PARTIAL** at RailCall the operator	Customer: `audit_chain` (§4.2). RailCall side: Render logs, reviewed on-demand.
§164.308(a)(2) Assigned security responsibility	**IMPLEMENTED**	Sami Ben Chaalia is the security officer. Contact: `security@railcall.ai`.
§164.308(a)(3-5) Workforce, training, incident procedures	**PARTIAL**	Two-person team; onboarding/training are informal but documented in this repo's CLAUDE.md and internal notes. Formal training program is on the roadmap when headcount grows.
§164.308(a)(6) Security incident procedures	**PARTIAL**	Incident response for a hosted-service outage: monitor Render, page Sami. For a customer-side incident: customer's own §164.308(a)(6) applies.
§164.308(a)(7) Contingency plan	**PARTIAL**	Gateway state is in Render's managed Postgres (their backup surface); station code is versioned in Git + published on GitHub Releases; issuer seed is backed up offline. No formal DR drill.
§164.308(a)(8) Evaluation	**IMPLEMENTED** at this document	This SRA is the periodic evaluation.

7. Physical safeguards (§164.310)

Not applicable at scale — RailCall does not operate physical infrastructure. The gateway runs on Render; Render's own physical safeguards documentation is the substantive answer here and is referenced by the BAA (Exhibit A, subcontractors row).

8. Adoption

ADOPTED 2026-07-21 by:

- [x] Sami Ben Chaalia (Security Officer) — date: 2026-07-21

Adoption confirms: Sami has read this document, agrees the controls listed as IMPLEMENTED are accurately implemented, agrees the gaps listed are the real gaps, and commits to re-verifying at each station release + at least annually.

Adoption does NOT mean: - RailCall is HIPAA-compliant. HIPAA has no certifying body; compliance is self-determined and enforced per-incident by OCR. - This SRA has been reviewed by legal counsel. Counsel review is required before this document is presented to a covered entity as evidence. - The BAA is executable. `legal/BAA_DRAFT.md` remains draft-only until counsel signs off.

Next review: at the next station release, and no later than 2027-07-21.

9. Revision history

Version	Date	Author	Change
v1	2026-07-21	Claude (drafted from Phase 1 evidence + P0-1 CLI remediation + paid-tier launch)	Initial draft against `railcall-engine@9521956fd`, `railcall-core@06bdd1449e`.